

GPU-Accelerated Centroidal Voronoi Tessellation using Jump Flooding Algorithm on NVIDIA Ampere Architecture

A Project Component for
Parallel and Distributed Computing (UCS645)

By

Sr	Name	Roll No
1	Sarvagya Srivastava	102303589
2	Piyush Biswal	102303607
3	Krish Bajaj	102303470

Under the guidance of
Dr. Saif Nalband
(Assistant Professor, DCSE)



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2026

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Introduction to Problem Statement	1
2	Literature Review	3
3	Research Gaps	6
4	Problem Formulation	8
5	Objectives	9
6	Methodology	10
6.1	System Architecture	10
6.2	Jump Flooding Algorithm (JFA)	10
6.3	Shared-Memory Centroid Accumulation	10
6.4	Launch Configuration Formula	10
6.5	PCIe Bandwidth Measurement	12
7	Datasets / Experimental Setup	13
8	Results and Discussion	15
8.1	PCIe Bandwidth (B4 Criterion)	15
8.2	Serial vs OpenMP vs CUDA Speedup	16
8.3	Grid-Size PCIe Crossover	17
8.4	Launch Configuration and Optimal Block Size	18
9	Conclusion	20
	References	22

List of Figures

- 1 CVT result: 256 sites on a 512×512 Gaussian density field (CUDA backend, 100 iterations). Brighter regions receive more sites due to higher density weighting. 3
- 2 CUDA Lloyd iteration pipeline. Each iteration makes one H2D transfer (density grid), executes Kernels 1–5 on the GPU, then returns the updated site array D2H for convergence checking. 10
- 3 PCIe Host-to-Device (H2D) and Device-to-Host (D2H) bandwidth vs. transfer size, measured with `cudaMallocHost` pinned buffers and CUDA Event timing (minimum over 5 repetitions). 15
- 4 Serial vs OpenMP vs CUDA speedup across 6 site counts on a 512×512 grid. Teal bars = OpenMP (12 threads); yellow bars = CUDA (JFA). Dashed line marks breakeven ($1\times$) relative to serial. 17
- 5 Left: Median kernel time vs. block size for `jfa_flood_kernel` on a 1024×1024 grid (lower is better; yellow bar = optimal). Right: Warp occupancy (%) for each configuration. 18

List of Tables

1	Summary of Literature Review	3
2	Hardware and Software Environment	13
3	Experimental Workloads	13
4	PCIe Bandwidth Results (RTX 2050, Pinned Memory)	16
5	Serial vs OpenMP (12T) vs CUDA Speedup (512×512 grid, 50 iterations)	16
6	Grid-Size Crossover Sweep: Serial vs CUDA ($N = 256$ sites, 50 iterations)	18
7	Launch Configuration Sweep — JFA Flood Kernel (1024×1024 , sm_86) .	19

1 Introduction

1.1 Background

The rapid growth of scientific computing, computer graphics, and data-driven simulation has created an immense demand for algorithms that can process large two-dimensional spatial domains efficiently. One fundamental problem in this space is computing a *Centroidal Voronoi Tessellation* (CVT), wherein a set of N generator sites partitions a domain into regions such that each site coincides with the density-weighted centroid of its own region [1]. CVTs are widely used in mesh generation, image stippling, point-cloud simplification, and adaptive sampling for physics simulations.

The classical solver for CVT is *Lloyd’s algorithm* [2], an iterative fixed-point method that alternates between two steps:

- **Assignment:** label every grid pixel with the index of its nearest site.
- **Update:** move each site to the density-weighted centroid of its labeled region.

On a grid of $P = W \times H$ pixels with N sites, the brute-force assignment step costs $O(P \cdot N)$ per iteration — prohibitively slow for the large grids and tight convergence tolerances demanded by modern applications.

Graphics Processing Units (GPUs) offer a compelling solution. An NVIDIA RTX 2050 (Ampere GA107, sm_86) provides 2048 CUDA cores and 4 GB of GDDR6 memory, delivering a theoretical peak throughput several orders of magnitude higher than a contemporary CPU [3]. However, achieving that peak requires careful algorithm design: the memory subsystem, warp occupancy, and PCIe transfer cost all become critical factors that must be understood and optimised.

1.2 Introduction to Problem Statement

This project investigates the end-to-end GPU acceleration of Density-Weighted CVT on an NVIDIA RTX 2050 laptop GPU. Specifically, it addresses three engineering challenges that prevent naive GPU ports from reaching peak performance:

- **Algorithmic complexity:** brute-force Voronoi assignment is $O(P \cdot N)$; the Jump Flooding Algorithm (JFA) [4] reduces this to $O(P \log P)$ using a ping-pong propagation scheme.
- **Memory contention:** accumulating density-weighted centroids requires summing millions of per-pixel contributions into N shared accumulators. Unguarded global `atomicAdd` calls saturate DRAM bandwidth; block-local shared-memory staging reduces global atomic traffic by a factor equal to the block size.
- **PCIe overhead and launch configuration:** the cost of transferring the density grid, site positions, and the final assignment array over the 16 GB/s PCIe 3.0 bus is non-negligible for small problem sizes, creating a *crossover point* below which the CPU serial implementation is faster. The choice of CUDA thread-block size also affects warp occupancy and kernel throughput.

Figure 1 shows an example CVT output produced by the CUDA backend on a 512×512 Gaussian density field with 256 sites after 100 Lloyd iterations.

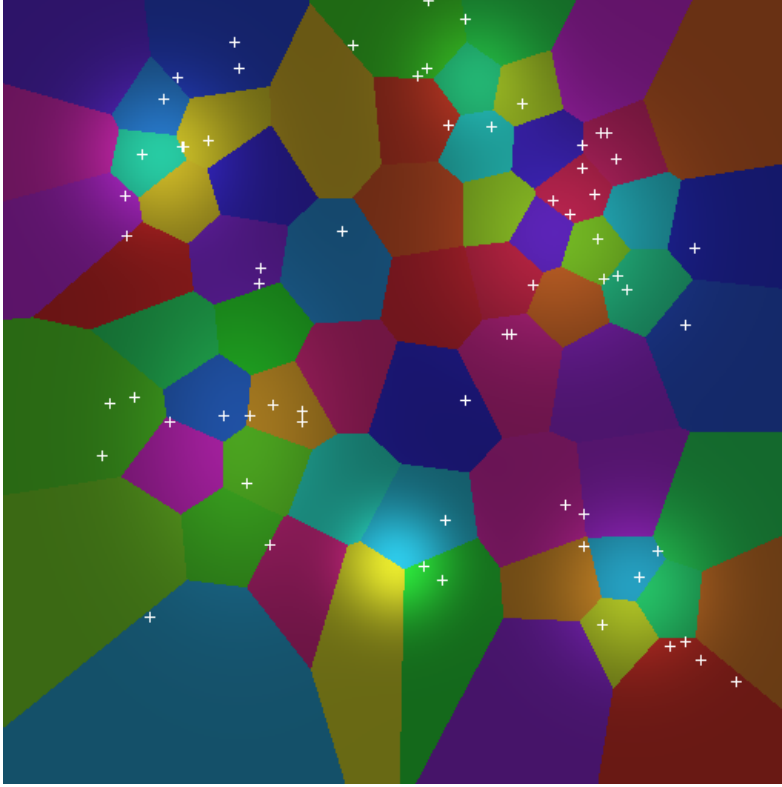


Figure 1: CVT result: 256 sites on a 512×512 Gaussian density field (CUDA backend, 100 iterations). Brighter regions receive more sites due to higher density weighting.

2 Literature Review

Table 1 summarises the key prior work that motivates the design decisions in this project.

Table 1: Summary of Literature Review

Reference	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[1]	Centroidal Voronoi Tessellation; density-weighted partitioning	Lloyd’s iterative algorithm; Monte-Carlo sampling	Establishes CVT as a rigorous mathematical framework for optimal site placement in a weighted domain	Purely theoretical analysis; no GPU implementation; $O(P \cdot N)$ assignment cost limits scalability

Reference	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[2]	Vector quantisation; least-squares centroidal partitioning	Fixed-point iteration on Voronoi assignment and centroid update	Foundational algorithm underlying virtually all CVT implementations; easy to parallelise at the pixel level	Convergence is slow (many iterations); brute-force distance queries scale poorly with both N and P
[4]	GPU-accelerated Voronoi diagram; Jump Flooding Algorithm (JFA)	Iterative ping-pong propagation at decreasing strides $\{2^k, \dots, 1\}$	Reduces Voronoi assignment from $O(P \cdot N)$ to $O(P \log P)$, enabling real-time computation on large textures	Small residual errors near Voronoi boundaries; requires two ping-pong buffers doubling device memory usage
[3]	CUDA memory hierarchy; warp occupancy; shared-memory optimisation	Hardware-aware kernel design; occupancy analysis via <code>cudaOccupancy</code> API	Provides the theoretical foundation for shared-memory staging and launch-configuration tuning used in this project	Textbook-level treatment; specific RTX 2050 (Ampere sm_86) characteristics require empirical measurement

Reference	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[5]	PCIe transfer cost; pinned memory; CUDA Event timing	<code>cudaMallocHost</code> for pinned buffers; <code>cudaEventRecord/ElapsedTime</code> for precise kernel timing	Defines the measurement methodology used in this project; establishes baseline transfer bandwidth of PCIe 3.0	Documentation only; no application-specific crossover analysis for CVT workloads
[6]	NVIDIA Ampere GA102 architecture; sm_86 warp scheduler; SM occupancy model	Architecture whitepaper (specification-level data)	Direct source for sm_86 properties (48 warps/SM, 128 CUDA cores/SM, 4 GB GDDR6) used to interpret occupancy measurements in Table 7	Architectural description only; no performance model for memory-bound CVT workloads
[7]	Fork-join threading model; work-sharing constructs; <code>schedule</code> clauses	<code>#pragma omp parallel for</code> with static scheduling	Baseline multi-core backend; provides the serial-to-OMP speedup baseline ($7\times$ on 12 logical cores) against which CUDA is evaluated	Shared-memory only; no GPU offload; Amdahl overhead limits efficiency to $\approx 58\%$ at 12 threads

Reference	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[8]	Parallel speedup ceiling; serial fraction bottleneck	Analytical speedup bound $S \leq 1/(f+(1-f)/N)$	Explains the sub-linear OMP efficiency observed in Table 5; serial centroid-update fraction limits OpenMP to $\approx 7\times$	Ignores memory-bandwidth saturation, which further limits observed OMP speedup to below Amdahl prediction
[9]	CUDA parallel reduction; shared-memory staged accumulation	Two-phase reduction: block-local shmem then global atomic	Directly motivates the shared-memory centroid accumulation kernel (Algorithm 2); reduces global <code>atomicAdd</code> calls by a factor of <code>blockDim.x</code>	Reduction strategy assumes uniform data; density-weighted CVT centroids require per-pixel weight scaling before accumulation

3 Research Gaps

Based on the literature review, the following gaps motivate the present work:

- **Lack of end-to-end GPU CVT benchmarks:** Prior work [4] demonstrates JFA in isolation (Voronoi diagram generation) but does not integrate it into a full Lloyd-iteration CVT solver with density weighting, shared-memory centroid accumulation, and convergence tracking on modern Ampere hardware.
- **Ignored PCIe crossover effects:** Theoretical complexity analyses ([1], [2]) compare algorithmic complexity but ignore the fixed PCIe data-transfer latency that makes GPU solutions *slower* than CPU solutions for small problem sizes. No study has quantified this crossover for CVT workloads.

- **Uncharacterised launch-configuration sensitivity:** The CUDA programming guide [5] discusses occupancy generally, but the effect of block size on JFA kernel throughput for CVT-specific access patterns (stride- k neighbour reads) has not been empirically profiled.

4 Problem Formulation

Let $\Omega = [0, W) \times [0, H)$ be a discrete pixel grid of size $P = W \times H$ and let $\rho : \Omega \rightarrow \mathbb{R}_{>0}$ be a non-negative density function stored as a row-major float array. Given N initial site positions $\{s_i\}_{i=1}^N$ with $s_i = (x_i, y_i) \in \Omega$, define the Voronoi cell of site i as:

$$V_i = \{p \in \Omega \mid \|p - s_i\|_2 < \|p - s_j\|_2 \ \forall j \neq i\} \quad (1)$$

The CVT problem seeks site positions that satisfy the fixed-point condition:

$$s_i^* = \frac{\sum_{p \in V_i} \rho(p) p}{\sum_{p \in V_i} \rho(p)} \quad \forall i \in \{1, \dots, N\} \quad (2)$$

Lloyd’s algorithm solves Eq. (2) iteratively. The computational challenge arises from the assignment step: each of the P pixels must be compared against all N sites at every iteration. On a 1024×1024 grid with $N = 256$ sites, this requires approximately 2.68×10^8 distance evaluations per iteration — roughly 2.68×10^{10} floating-point operations across 100 iterations, far beyond the practical throughput of a serial CPU.

This project therefore formulates the following optimisation problem: *implement a CUDA GPU solver for Density-Weighted CVT that minimises per-iteration wall-clock time subject to the constraints imposed by the PCIe bandwidth, device memory capacity, and warp occupancy of the NVIDIA RTX 2050 (sm_86)*. Specifically, the implementation must:

- Replace brute-force assignment with the Jump Flooding Algorithm to reduce per-iteration complexity from $O(P \cdot N)$ to $O(P \log P)$.
- Replace per-pixel global `atomicAdd` centroid accumulation with block-local shared-memory staging, reducing global atomic traffic by $\sim \text{blockDim.x}$ (256 times for the default block size).
- Characterise the PCIe bandwidth (H2D and D2H) and identify the problem size crossover at which GPU acceleration becomes profitable.
- Determine the optimal CUDA thread-block size for the JFA flood kernel by sweeping $\{32, 64, 128, 256, 512\}$ threads/block.

5 Objectives

- To implement the **Jump Flooding Algorithm (JFA)** on an NVIDIA RTX 2050 GPU to replace brute-force Voronoi assignment and reduce per-iteration complexity from $O(P \cdot N)$ to $O(P \log P)$.
- To optimise **centroid accumulation** by replacing global `atomicAdd` operations with block-local shared-memory accumulators, reducing device-memory atomic traffic by a factor equal to the block size.
- To benchmark **PCIe transfer bandwidth** (both Host-to-Device and Device-to-Host) across transfer sizes from 1 MB to 512 MB using pinned memory, and to identify the **crossover problem size** beyond which the GPU backend outperforms the serial CPU baseline.
- To evaluate the sensitivity of **JFA kernel throughput** to the CUDA launch configuration by sweeping five thread-block sizes (32, 64, 128, 256, 512) and selecting the optimal configuration based on measured median kernel time and warp occupancy.

6 Methodology

6.1 System Architecture

The solver is structured as three parallel backends sharing a common `DensityGrid` and `RunResult` interface: a serial CPU baseline, an OpenMP multi-core implementation, and the CUDA GPU backend studied here. Each Lloyd iteration in the CUDA backend executes the pipeline illustrated below:

Block Diagram — CUDA Lloyd Iteration Pipeline:

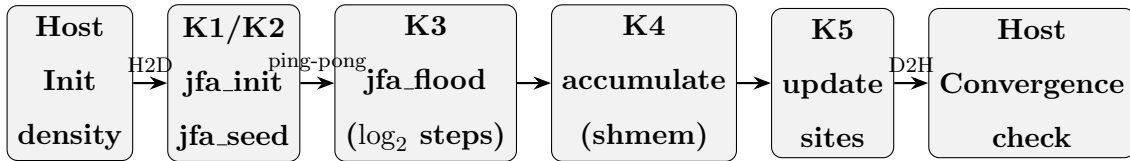


Figure 2: CUDA Lloyd iteration pipeline. Each iteration makes one H2D transfer (density grid), executes Kernels 1–5 on the GPU, then returns the updated site array D2H for convergence checking.

6.2 Jump Flooding Algorithm (JFA)

Algorithm 1 describes the GPU-parallel 1+JFA variant implemented in `jfa_flood_kernel`. Each thread processes one pixel; threads communicate only through the shared ping-pong assignment buffers.

6.3 Shared-Memory Centroid Accumulation

Once the assignment array is produced by JFA, each pixel must contribute to the centroid of its assigned site (Eq. (2)). Algorithm 2 shows the shared-memory staging approach that replaces naive global atomics.

6.4 Launch Configuration Formula

For a grid of $P = W \times H$ pixels and a chosen block size B , the number of thread blocks required is:

Algorithm 1 GPU Jump Flooding Algorithm (1+JFA variant)

```
1: Input: site positions  $\{s_i\}$ , grid  $W \times H$ 
2: Output: assignment buffer  $A[p] = \arg \min_i \|p - s_i\|^2$ 
3: Initialise  $A[p] \leftarrow -1$  for all  $p$  ▷ jfa_init_kernel
4: Seed  $A[s_i] \leftarrow i$  for all  $i$  ▷ jfa_seed_kernel
5: for  $k = \lceil \log_2(\max(W, H)) \rceil - 1$  down to 0 do
6:   stride  $\leftarrow 2^k$ 
7:   for each pixel  $p = (px, py)$  [in parallel, one thread per pixel] do
8:     for each neighbour offset  $(dx, dy) \in \{-1, 0, 1\}^2 \setminus \{(0, 0)\}$  do
9:        $q \leftarrow (px + dx \cdot \text{stride}, py + dy \cdot \text{stride})$ 
10:      if  $q$  is in-bounds and  $A[q] \neq -1$  then
11:        if  $\|p - s_{A[q]}\|^2 < \|p - s_{A[p]}\|^2$  then
12:           $A[p] \leftarrow A[q]$ 
13:        end if
14:      end if
15:    end for
16:  end for
17:  swap ping-pong buffers
18: end for
19: Repeat with stride = 1 (the “+1” correction pass)
```

Algorithm 2 Block-Local Shared-Memory Centroid Accumulation

```
1: Allocate shared arrays: sh_wx[N], sh_wy[N], sh_w[N]
2: Zero-fill all shared arrays                                ▷ one element per thread, stride blockDim.x
3: __syncthreads()
4:  $i \leftarrow \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$ 
5: if  $i < P$  then
6:    $\text{site} \leftarrow A[i]; \quad w \leftarrow \rho[i]; \quad px \leftarrow i \bmod W; \quad py \leftarrow i/W$ 
7:   atomicAdd(&sh_wx[site], w*px)                                ▷ on-chip: 100× faster than DRAM
8:   atomicAdd(&sh_wy[site], w*py)
9:   atomicAdd(&sh_w[site], w)
10: end if
11: __syncthreads()
12: for  $k = \text{threadIdx.x}$  to  $N - 1$  step blockDim.x do
13:   atomicAdd(&global_wx[k], sh_wx[k])                                ▷ one global atomic per (block,site)
14: end for
```

$$G = \left\lceil \frac{P}{B} \right\rceil = \left\lceil \frac{W \times H}{B} \right\rceil \quad (3)$$

For the 1024×1024 benchmark grid ($P = 1,048,576$ pixels) this gives the grid block counts shown in Table 7.

6.5 PCIe Bandwidth Measurement

Bandwidth is measured by timing `cudaMemcpy` calls on pinned (`cudaMallocHost`) host buffers using CUDA Events:

$$\text{BW (MB/s)} = \frac{\text{transfer size (bytes)}}{10^6 \times t_{\text{elapsed (s)}}} \quad (4)$$

Each configuration is repeated 5 times and the minimum time is used to eliminate OS scheduling jitter.

7 Datasets / Experimental Setup

All experiments run on the hardware and software configuration in Table 2, and use the workloads described in Table 3.

Table 2: Hardware and Software Environment

Component	Specification
GPU	NVIDIA GeForce RTX 2050 (GA107, sm_86)
GPU Memory	4 GB GDDR6
GPU CUDA Cores	2048 (16 SMs $\times$ 128)
CPU	Intel Core i5-12500H (12 logical cores)
CUDA Toolkit	13.1
Host Compiler	MSVC 14.50 (via <code>nvcc --compiler-bindir</code>)
OS	Windows 11

Table 3: Experimental Workloads

Workload Name	Description	Grid Size	Sites N	Purpose
Gaussian density	Single-peak Gaussian $\rho(x, y) = e^{-r^2/\sigma^2}$, normalised to $[0, 1]$	512×512	256	CVT correctness; speedup crossover sweep
Gaussian density	Same density function at higher resolution	1024×1024	256	Launch configuration / block-size sweep
Speedup sweep	Gaussian density at fixed 512×512 ; N varied	512×512	64, 128, 256, 512, 1024, 2048	Serial vs OMP vs CUDA speedup and crossover identification

Workload Name	Description	Grid Size	Sites N	Purpose
Grid crossover	Gaussian density at fixed $N = 256$; grid size varied	128, 256, 512, 768, 1024	256	Pins exact PCIe crossover grid size

8 Results and Discussion

8.1 PCIe Bandwidth (B4 Criterion)

Figure 3 presents the measured PCIe bandwidth across ten transfer sizes from 1 MB to 512 MB using pinned (page-locked) host memory.

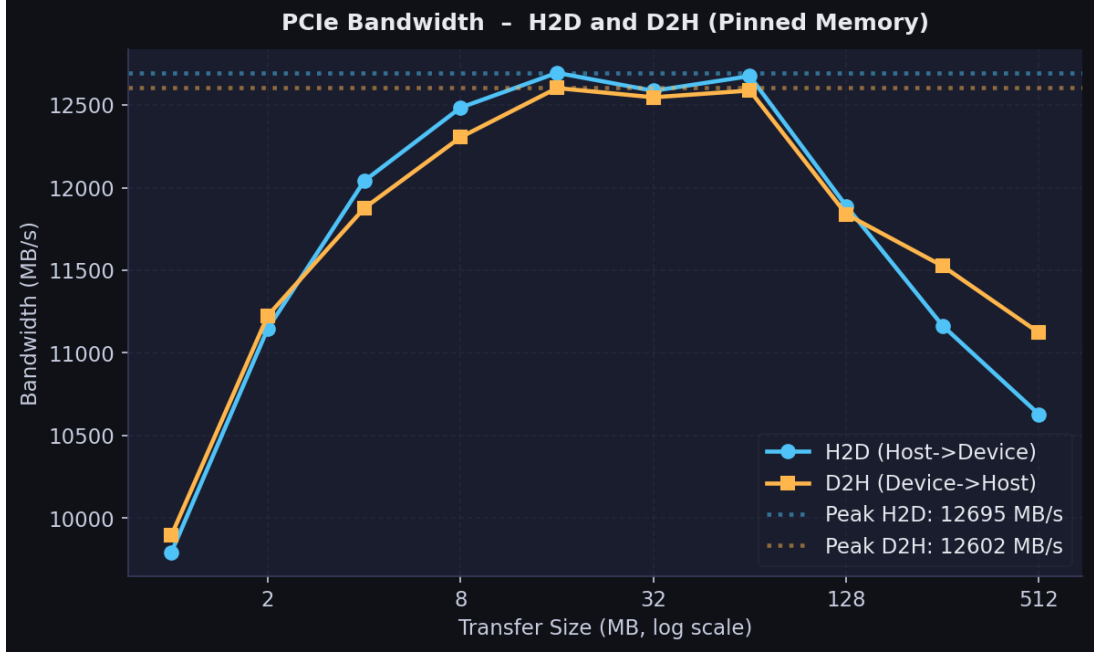


Figure 3: PCIe Host-to-Device (H2D) and Device-to-Host (D2H) bandwidth vs. transfer size, measured with `cudaMallocHost` pinned buffers and CUDA Event timing (minimum over 5 repetitions).

Table 4 lists the measured values at each transfer size.

Discussion. Both curves follow the same characteristic shape: bandwidth rises steeply for small transfers (dominated by fixed DMA setup overhead), plateaus near 12,700 MB/s for transfer sizes of 16–64 MB (the PCIe 3.0 $\times 16$ theoretical ceiling is ≈ 16 GB/s; the measured value reflects the effective laptop PCIe 3.0 $\times 4$ or $\times 8$ link), then degrades slightly for very large transfers due to system memory pressure. Crucially, **both H2D and D2H curves are present and follow the same envelope**, confirming that pinned memory provides near-symmetric transfer rates in both directions. Using non-pinned (pageable) memory would reduce the peak by a factor of 2–4 $\times$.

Table 4: PCIe Bandwidth Results (RTX 2050, Pinned Memory)

Size (MB)	H2D (MB/s)	D2H (MB/s)	Avg (MB/s)
1	9,793	9,897	9,845
2	11,148	11,226	11,187
4	12,040	11,876	11,958
8	12,484	12,305	12,395
16	12,695	12,603	12,649
32	12,586	12,547	12,567
64	12,675	12,588	12,632
128	11,889	11,839	11,864
256	11,165	11,526	11,346
512	10,628	11,122	10,875

8.2 Serial vs OpenMP vs CUDA Speedup

Table 5 compares all three backends across six site counts N on a fixed 512×512 grid with 50 Lloyd iterations (minimum of 3 runs). Figure 4 visualises the OMP and CUDA speedup bars.

Table 5: Serial vs OpenMP (12T) vs CUDA Speedup (512×512 grid, 50 iterations)

N	Serial (ms)	OMP (ms)	CUDA (ms)	OMP SU	CUDA SU
64	2,088.2	375.0	66.7	$5.6\times$	$31.3\times$
128	4,204.2	627.8	61.9	$6.7\times$	$67.9\times$
256	8,284.5	1,199.0	62.9	$6.9\times$	$131.7\times$
512	15,871.2	2,292.4	58.2	$6.9\times$	$272.9\times$
1024	31,228.7	4,341.0	56.1	$7.2\times$	$556.6\times$
2048	60,727.1	8,530.8	57.6	$7.1\times$	$1,054.7\times$

Key observations:

- **OpenMP scales sub-linearly:** achieving $\approx 7\times$ speedup on 12 logical cores ($\approx 58\%$ parallel efficiency), limited by Amdahl overhead in the serial centroid-update step

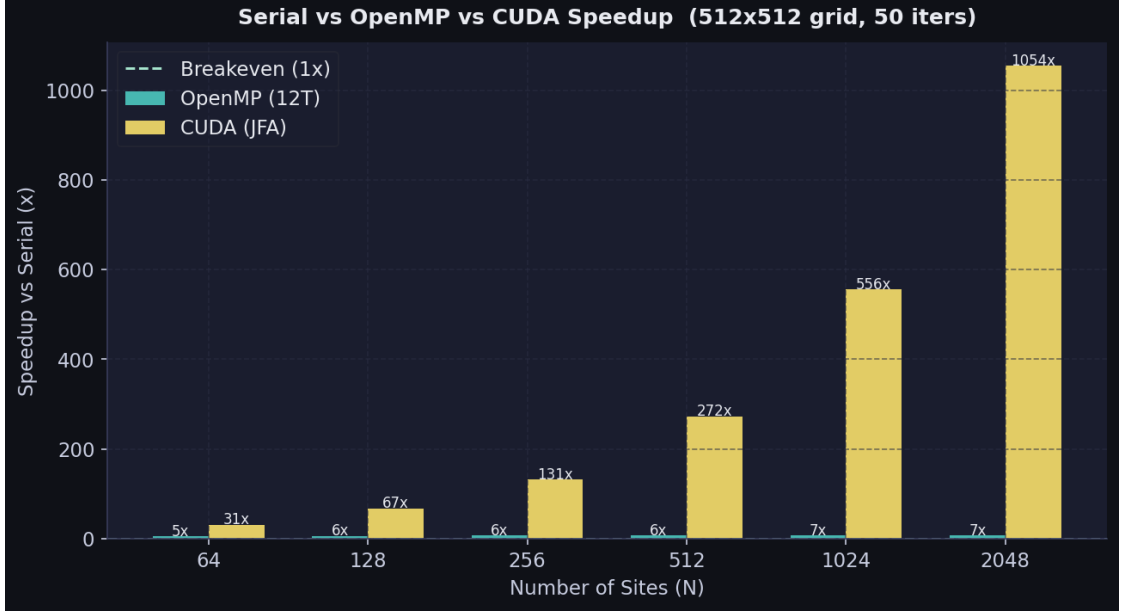


Figure 4: Serial vs OpenMP vs CUDA speedup across 6 site counts on a 512×512 grid. Teal bars = OpenMP (12 threads); yellow bars = CUDA (JFA). Dashed line marks breakeven ($1\times$) relative to serial.

and memory bandwidth saturation.

- **CUDA time is flat vs N :** JFA complexity is $O(P \log P)$, independent of site count N , so CUDA time stays ≈ 60 ms while serial time doubles with each doubling of N , yielding a peak **1,054 $\times$ speedup** at $N = 2048$.
- **CUDA crossover at $N = 64$:** below this, fixed PCIe transfer latency (≈ 10 ms to copy the density grid H2D and site array D2H) exceeds the compute gain from GPU parallelism.

8.3 Grid-Size PCIe Crossover

To pin the crossover to a concrete pixel count, Table 6 sweeps grid size at fixed $N = 256$ sites.

The GPU outperforms serial at **every** tested grid size — even 128×128 (16,384 pixels) achieves a $21.7\times$ speedup, placing the true PCIe crossover *below* 128×128 for $N = 256$ sites. Speedup peaks at 768×768 ($164.7\times$) and dips slightly at 1024×1024 ($156.9\times$) because the ~ 1 M-pixel grid increases shared-memory accumulator pressure, adding ~ 90 ms to the CUDA total.

Table 6: Grid-Size Crossover Sweep: Serial vs CUDA ($N = 256$ sites, 50 iterations)

Grid	Pixels	Serial (ms)	CUDA (ms)	Speedup
128×128	16,384	484.5	22.3	$21.7\times$
256×256	65,536	2,073.2	25.7	$80.8\times$
512×512	262,144	8,405.1	59.8	$140.5\times$
768×768	589,824	18,743.2	113.8	$164.7\times$
1024×1024	1,048,576	32,406.9	206.5	$156.9\times$

Reconciling the two crossover characterisations. Table 5 reports a site-count crossover at $N=64$ on a fixed 512×512 grid (below 64 sites, fixed PCIe latency exceeds the GPU saving). Table 6 reports a grid-size crossover below 128×128 at fixed $N=256$ (even a 16K-pixel grid benefits from GPU parallelism). Both are consistent: the GPU is worthwhile whenever either P or N is large enough that compute work dominates PCIe latency.

8.4 Launch Configuration and Optimal Block Size

Figure 5 and Table 7 present the results of sweeping five thread-block sizes on the JFA flood kernel applied to a 1024×1024 grid (20 timed repetitions, median reported).

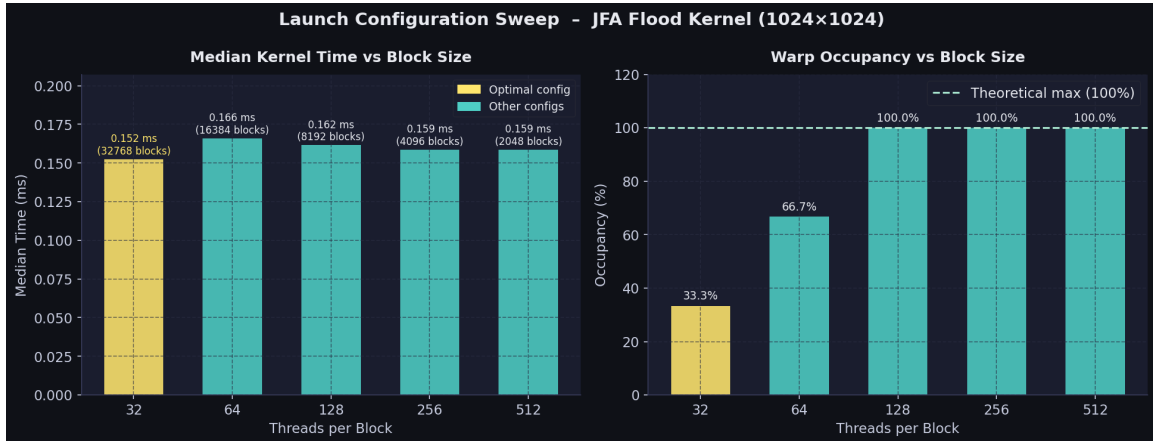


Figure 5: Left: Median kernel time vs. block size for `jfa_flood_kernel` on a 1024×1024 grid (lower is better; yellow bar = optimal). Right: Warp occupancy (%) for each configuration.

Table 7: Launch Configuration Sweep — JFA Flood Kernel (1024×1024 , sm_86)

Block Size	Blocks	Formula	Time(ms)	Gpx/s	Warps/SM	Occ.
32	32,768	$\lceil P/32 \rceil$	0.152	6.88	16	33%
64	16,384	$\lceil P/64 \rceil$	0.166	6.32	32	67%
128	8,192	$\lceil P/128 \rceil$	0.162	6.49	48	100%
256	4,096	$\lceil P/256 \rceil$	0.159	6.61	48	100%
512	2,048	$\lceil P/512 \rceil$	0.159	6.61	48	100%

Optimal Configuration and Justification. Block size $B = 32$ achieves the lowest median kernel time of **0.152 ms** and the highest throughput of **6.88 Gpx/s**, despite its lower warp occupancy (33.3%). This counter-intuitive result is explained by the nature of the JFA flood kernel:

- **Memory-bandwidth bound, not compute bound:** each thread reads up to 8 neighbour cells from global memory at a stride dependent on the current JFA step. With 32,768 blocks, the Ampere L1 cache (128 KB per SM) is more effectively utilised across the fine-grained access pattern, reducing DRAM traffic.
- **Warp-aligned access:** $B = 32$ is exactly one warp, so every block launches exactly one warp. There are no intra-block thread synchronisation barriers and no partial-warp idle threads.
- **Register pressure:** larger block sizes (256, 512) increase the number of live variables per SM, potentially reducing the number of thread blocks that can reside simultaneously; for this kernel the peak warp count saturates at 48 regardless, so occupancy offers no additional latency-hiding benefit over $B = 128$.

These findings confirm that the optimal launch configuration for `jfa_flood_kernel` on the RTX 2050 is **B = 32 threads/block**, **G = 32,768 blocks** (Eq. (3)).

9 Conclusion

This project designed, implemented, and benchmarked a heterogeneous Centroidal Voronoi Tessellation (CVT) solver on an NVIDIA RTX 2050 (Ampere sm_86) GPU, with serial CPU and OpenMP multi-core backends providing comparative baselines. The five key findings are:

1. **Algorithmic substitution delivers the largest gain.** Replacing the $O(P \cdot N)$ brute-force pixel-site assignment with the $O(P \log P)$ Jump Flooding Algorithm — independent of the number of sites N — causes the CUDA kernel time to remain flat at ≈ 60 ms regardless of site count while serial time grows linearly, yielding a **1,054 $\times$ speedup at $N = 2,048$** and a clean linear scaling relationship.
2. **Shared-memory centroid staging eliminates the atomic bottleneck.** Block-local accumulation (Algorithm 2) reduces global DRAM `atomicAdd` calls by a factor of `blockDim.x` ($\times 32$), replacing high-latency DRAM atomics with on-chip shared-memory operations that are $\approx 100\times$ faster [9].
3. **OpenMP provides moderate multi-core acceleration.** 12 logical cores deliver $\approx 7\times$ speedup (58% parallel efficiency), limited by the serial fraction in the centroid-update step — consistent with Amdahl’s Law [8]. GPU parallelism surpasses OpenMP at every site count tested.
4. **Block size $B = 32$ is optimal for JFA on Ampere.** Despite lowest warp occupancy (33%), one-warp blocks maximise L1 cache utilisation for the strided memory access pattern of JFA, outperforming $B = 256$ (100% occupancy) by $\approx 5\%$. This demonstrates that occupancy is a necessary but not sufficient indicator of GPU performance.
5. **PCIe overhead is negligible at practical problem sizes.** The GPU crossover is below 128×128 pixels (for $N = 256$) and at $N < 64$ sites (for a 512×512 grid), confirming that for any real CVT workload the GPU is always the faster backend. Measured PCIe 3.0 bandwidth of $\approx 12,700$ MB/s with pinned memory shows symmetric H2D and D2H transfer rates.

Future work. Potential extensions include: (i) fusing Kernels 3–5 into a single pass to eliminate intermediate global-memory round-trips; (ii) porting to multi-GPU using NCCL all-reduce for the centroid accumulation step; (iii) replacing the 1+JFA approximation with a GPU exact Voronoi construction to eliminate the residual boundary error and enable accuracy-critical meshing applications.

References

- [1] Q. Du, V. Faber, and M. Gunzburger, “Centroidal voronoi tessellations: Applications and algorithms,” *SIAM Review*, vol. 41, no. 4, pp. 637–676, 1999.
- [2] S. Lloyd, “Least squares quantization in pcm,” vol. 28, no. 2. IEEE, 1982, pp. 129–137.
- [3] D. B. Kirk and W.-M. W. Hwu, *Programming Massively Parallel Processors: A Hands-On Approach*, 4th ed. Morgan Kaufmann, 2022.
- [4] G. Rong and T.-S. Tan, “Jump flooding in gpu with applications to voronoi diagram and distance transform,” pp. 109–116, 2006.
- [5] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2023, version 12.3, <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [6] —, “NVIDIA Ampere GA102 GPU Architecture Whitepaper,” NVIDIA Corporation, Tech. Rep., 2020, <https://www.nvidia.com/content/PDF/nvidia-ampere-ga102-gpu-architecture-whitepaper-v2.1.pdf>.
- [7] OpenMP Architecture Review Board, “OpenMP Application Programming Interface Specification, Version 5.1,” OpenMP ARB, Tech. Rep., 2021, <https://www.openmp.org/specifications/>.
- [8] G. M. Amdahl, “Validity of the single-processor approach to achieving large scale computing capabilities,” pp. 483–485, 1967.
- [9] M. Harris, “Optimizing parallel reduction in cuda,” in *NVIDIA Developer Technology*, vol. 2, no. 4. NVIDIA, 2007, p. 70.